

ARIA

Intelligent Insurance Bundle Recommendation Technical Report

Team Members

Mohamed Achref Hemissi

Mohamed Dhia Medini

Rayen Chemlali

Leith Engazzou

Final Score: 0.4875

Macro F1: 0.5577

Model: LightGBM (75 trees, 10 classes)

Features: 62 engineered features

Latency: 1.18 s

Model Size: 1.86 MB

Date: February 22, 2026

Contents

1	Problem Statement and Constraints	3
1.1	Task Definition	3
1.2	Scoring Formula	3
1.3	Resource Constraints	3
2	Data Overview	3
2.1	Dataset Summary	3
2.2	The Imbalance Problem	3
3	Feature Engineering — 62 Features	4
3.1	Pipeline Overview	4
3.2	Engineered Features (17 New Columns)	5
3.2.1	Household and Financial Features	5
3.2.2	Risk and Behavioural Features	5
3.2.3	Missingness Indicators	5
3.2.4	Temporal Cyclical Encoding	6
3.3	Key Innovation — Multi-Class Target Encoding (v7)	6
3.4	Final Feature Matrix	6
4	Class Imbalance Strategy	7
4.1	3-Step Resampling Pipeline	7
4.2	Why <code>class_weight=None</code> (Not 'balanced')	7
5	Model Architecture — LightGBM	7
5.1	Why LightGBM?	7
5.2	Final Hyperparameters	8
5.3	Number of Trees — 75 is the Sweet Spot	8
5.4	Prediction Strategy: Raw Argmax	8
6	Hyperparameter Tuning — 4-Phase Sequential Search	8
6.1	Phase 1 — Encoding Comparison	9
6.2	Phase 2 — Class Weight	9
6.3	Phase 3 — HP Grid Search (75 Trees Fixed)	9
6.4	Phase 4 — Feature Selection	9
6.5	Combined Impact Summary	9

7	Explainability Analysis	9
7.1	Three Methods Compared	9
7.2	SHAP Visualisations Produced	10
7.3	Key Explainability Findings	10
8	Results — Competition Leaderboard	10
8.1	Submission History	10
8.2	Per-Class OOF F1 (v7 Best Model)	10
9	Score Decomposition	10
10	What Didn't Work — Lessons Learned	11
10.1	Optimising Latency/Size at the Cost of F1	11
10.2	<code>class_weight='balanced'</code> with SMOTE	11
10.3	Per-Class Threshold Tuning (v1)	12
10.4	OOF vs. Full-Train Encoding Inconsistency	12
10.5	Alternative Models Tested	12
11	Conclusion	12
11.1	Summary of Contributions	12
11.2	End-to-End Architecture	12
11.3	Reproducibility	13

1 Problem Statement and Constraints

1.1 Task Definition

Predict which of **10 insurance bundles** (classes 0–9) a customer will purchase, evaluated by **Macro F1-Score** — every class counts equally, regardless of support.

1.2 Scoring Formula

The competition score penalises both model size and inference latency:

$$\text{score} = \text{Macro_F1} \times \max(0.5, 1 - \frac{\text{size_MB}}{200}) \times \max(0.5, 1 - \frac{\text{latency_s}}{10})$$

A perfect F1 model at 50 MB and 1 s latency would score only **0.675** — both accuracy *and* efficiency matter.

1.3 Resource Constraints

Table 1: Competition Resource Limits

Constraint	Limit
Model ZIP size	50 MB
RAM at inference	1 GB
CPU cores	1
Prediction latency	10 s
Total runtime	120 s

2 Data Overview

2.1 Dataset Summary

Table 2: Dataset Characteristics

Attribute	Train	Test
Rows	60,868	15,218
Raw columns	29 (incl. target)	28
Duplicate rows	0	0
Columns with missing values	6	5

2.2 The Imbalance Problem

The dataset exhibits **extreme class imbalance** — a $7,227\times$ ratio between the largest and smallest class.

Table 3: Target Class Distribution (Training Set)

Class	Bundle Name	Count	Share
C2	Basic_Health	36,136	59.4%
C4	Health_Dental_Vision	13,958	22.9%
C3	Family_Comprehensive	4,831	7.9%
C7	Premium_Health_Life	2,286	3.8%
C1	Auto_Liability_Basic	1,625	2.7%
C0	Auto_Comprehensive	823	1.4%
C6	Home_Standard	719	1.2%
C5	Home_Premium	479	0.8%
C8	Renter_Basic	6	0.01%
C9	Renter_Premium	5	0.008%

Macro F1 scores every class equally, so a naive majority-class model gets near-zero despite 59% accuracy. Two distinct sub-problems emerge:

- **Problem A — Moderate minorities** (C0, C5, C6): 479–823 samples. Addressable with SMOTE.
- **Problem B — Extreme minorities** (C8, C9): 5–6 samples total. A data ceiling — only broker encoding provides viable signal.

3 Feature Engineering — 62 Features

Starting from 28 raw columns, our pipeline produces **62 features** through creative engineering across four categories.

3.1 Pipeline Overview

Table 4: Feature Pipeline Stages

Stage	Features	Description
Raw columns (after 3 drops)	25	Dropped <code>User_ID</code> , <code>Payment_Schedule</code> , <code>Policy_Start_Day</code>
Engineered features	+17	Household, financial, risk, temporal, missingness
One-hot encoding	+7	Employment (3 dummies), Acquisition (4 dummies)
Target encoding (single)	+1	<code>Employer_ID_TargetEnc</code>
Multi-class target encoding	+20	<code>Broker_C0–C9</code> , <code>Region_C0–C9</code>
Dropped (replaced by encodings)	−8	Raw categoricals
Total	62	

3.2 Engineered Features (17 New Columns)

3.2.1 Household and Financial Features

Table 5: Household and Financial Engineered Features

Feature	Formula	Rationale
Total_Dependents	Adult + Child + Infant	Household size drives bundle choice
Has_Children	$(\text{Child} + \text{Infant}) > 0$	Family buyers are a distinct segment
log_Income	$\log(1 + \text{Income})$	Raw income is 48× skewed; log normalises
log_Income_Per_Dep	$\log(1 + \frac{\text{Income}}{\text{Deps}+1})$	Per-person affordability proxy

3.2.2 Risk and Behavioural Features

Table 6: Risk and Behavioural Engineered Features

Feature	Formula	Rationale
Risk_Score	$\text{Claims} - \text{Years_Clean} + \text{Grace_Ext}$	Composite financial stress index
Underwriting_Delayed	$\text{Processing_Days} > 0$	Binary flag (median = 0)
Fast_Buyer	$\text{Days_Since_Quote} < 7$	Fast deciders → Basic_Health
Indecision_Score	$\text{Days_Quote} \times \text{Amendments}$	Deliberation × complexity
Policy_Complexity	$\text{Riders} + \text{Amendments}$	Complex needs signal

3.2.3 Missingness Indicators

Table 7: Missingness Indicator Features

Feature	Formula	Rationale
Employer_ID_Present	$\text{Employer_ID.notna}()$	5.7% with employer = group policies
Broker_ID_Missing	$\text{Broker_ID.isna}()$	Structurally different from known brokers

Table 8: Temporal Cyclical Features

Feature	Formula	Rationale
Month_Sin / Month_Cos	$\sin / \cos(2\pi \cdot \text{Month}/12)$	December $\rightarrow$ January continuity
Week_Sin / Week_Cos	$\sin / \cos(2\pi \cdot \text{Week}/52)$	Week-of-year cyclical pattern

3.2.4 Temporal Cyclical Encoding

3.3 Key Innovation — Multi-Class Target Encoding (v7)

Standard target encoding collapses broker specialisation into a single mean. But brokers **specialise**: Broker 1.0 sells almost exclusively Basic_Health ($\sigma = 0.21$), while Broker 250 focuses on premium bundles.

Multi-class encoding replaces each high-cardinality categorical with **10 probability features** — one per bundle class:

$$P(\text{bundle} = k \mid \text{broker} = b) = \frac{n_{bk} \cdot P(k|b) + \lambda \cdot \text{prior}_k}{n_{bk} + \lambda}, \quad \lambda = 300$$

This gives the model the **full specialisation distribution** per broker/region, not just the weighted average.

Table 9: Multi-Class Target Encoding Impact

Encoding Strategy	Features	OOF Macro F1
Single-number target encoding	44	0.5977
Multi-class encoding (v7)	62	0.6418 (+0.044)

3.4 Final Feature Matrix

Table 10: Complete Feature Matrix (62 Features)

Feature Group	Count	Description
Original numeric	32	Policy, risk, temporal, financial
One-hot dummies	7	Employment (3), Acquisition (4)
Binary flags	3	Agency_National, Employer_ID_Present, Broker_ID_Missing
Employer_ID_TargetEnc	1	Smoothed single-number encoding
Broker_C0–Broker_C9	10	Multi-class broker probabilities
Region_C0–Region_C9	10	Multi-class region probabilities
Total	62	

4 Class Imbalance Strategy

4.1 3-Step Resampling Pipeline

Applied **inside each CV fold** — validation data is never touched to prevent leakage.

Table 11: 3-Step Resampling Pipeline

Step	Classes	Method	Before	After	Rationale
1	C8, C9	RandomOverSampler	5–6	200	SMOTE needs $k + 1$ samples minimum
2	C8, C9	SMOTE ($k = 3$)	200	600	Synthetic neighbours now viable
3	C0, C5, C6	SMOTE ($k = 5$)	479–823	3,000	Moderate minorities brought up

4.2 Why `class_weight=None` (Not `'balanced'`)

This was the **single largest improvement** in v7: **+0.022 OOF F1**.

When SMOTE already rebalances the training set, adding `class_weight='balanced'` **double-penalises** the majority class — once through SMOTE and again through gradient upweighting. Setting `class_weight=None` lets the resampled distribution speak for itself.

Table 12: Class Weight Ablation Study

<code>class_weight</code>	OOF Macro F1
<code>'balanced'</code>	0.5977
Manual weights	0.5715
None	0.6199 (+0.022)

5 Model Architecture — LightGBM

5.1 Why LightGBM?

Table 13: Model Selection Rationale

Constraint	LightGBM Advantage	Alternative Issue
50 MB ZIP limit	1.86 MB model file	Deep learning exceeds limit
10s predict latency	75 trees, ~0.12s locally	Random Forest too slow
1 GB RAM	Peak < 200 MB	CatBoost larger files
Macro F1 objective	Multiclass + SMOTE pre-balancing	XGBoost comparable but larger

Table 14: LightGBM Hyperparameter Configuration

Parameter	Value
objective	multiclass
num_class	10
class_weight	None (SMOTE handles imbalance)
n_estimators	75
learning_rate	0.07
max_depth	7
num_leaves	95
min_child_samples	20
subsample	0.8
colsample_bytree	0.8
reg_alpha	0.1
reg_lambda	0.5
random_state	42
n_jobs	1

5.2 Final Hyperparameters

5.3 Number of Trees — 75 is the Sweet Spot

Table 15: Tree Count Ablation

Trees	OOF F1	Local Latency	Notes
50	0.4487	0.47 s	Too few — F1 collapsed for minorities
75	0.5977	0.62 s	Best F1/latency balance
150	~0.58	1.0+ s	Diminishing F1, double latency

5.4 Prediction Strategy: Raw Argmax

v1 tuned per-class thresholds on OOF folds, inflating training F1 to 0.74 — but actual competition F1 was only 0.4719. Thresholds memorised fold-level noise. **Raw argmax** is more robust on unseen data.

6 Hyperparameter Tuning — 4-Phase Sequential Search

Each phase isolated one variable to measure its impact clearly.

Table 16: Phase 1: Encoding Strategy Comparison

Encoding	Features	OOF F1
Single-number	44	0.5898
Multi-class	62	0.5977 (+0.008)

6.1 Phase 1 — Encoding Comparison

6.2 Phase 2 — Class Weight

Table 17: Phase 2: Class Weight Comparison

class_weight	OOF F1
'balanced'	0.5977
None	0.6199 (+0.022)

6.3 Phase 3 — HP Grid Search (75 Trees Fixed)

Table 18: Phase 3: Top Grid Search Results

OOF F1	num_leaves	lr	min_child	reg_alpha	reg_lambda
0.6311	95	0.07	20	0.1	0.5
0.6306	95	0.07	20	0.1	2.0
0.6272	63	0.07	20	0.1	0.5
0.6250	31	0.07	20	0.05	2.0

6.4 Phase 4 — Feature Selection

All 62 features contributed at least one tree split. Feature set kept intact.

6.5 Combined Impact Summary

7 Explainability Analysis

Three complementary importance methods were used to validate the model’s decision logic.

7.1 Three Methods Compared

All three methods agree on the **top 4 features** — strong cross-validation of model logic.

Table 19: Cumulative Tuning Impact

Technique	Δ OOF F1
Multi-class target encoding	+0.008
<code>class_weight=None</code>	+0.022
Hyperparameter grid search	+0.013
Full-train encoding consistency	+0.004 (actual F1)
Combined (v7)	0.6418 OOF F1

Table 20: Feature Importance — Top 5 Features Across Three Methods

Rank	LightGBM Built-in	SHAP (mean SHAP)	Permutation (F1 drop)
1	Agency_National	Agency_National (0.306)	Agency_National
2	log_Income	log_Income (0.214)	log_Income
3	Broker_C1	Broker_C1 (0.181)	Broker_C1
4	Total_Dependents	Total_Dependents (0.178)	Total_Dependents
5	Broker_C6	Broker_C6 (0.115)	Deductible_Tier_Ord

7.2 SHAP Visualisations Produced

7.3 Key Explainability Findings

1. **Broker specialisation is the dominant signal.** 6 of the top 15 features are broker-related. Multi-class encoding captures the full specialisation distribution.
2. **Income drives the premium vs. basic split.** `log_Income` separates premium bundles from basic ones. SHAP shows a threshold effect around $\log(\text{income}) \approx 10.5$.
3. **Family size drives family-specific bundles.** `Total_Dependents` and `Has_Children` separate `Family_Comprehensive` from individual bundles.
4. **Minority classes C8/C9 are broker-dependent.** Without multi-class broker encoding, these 5–6 sample classes are unlearnable.
5. **All three importance methods agree** on the top features, validating the model’s decision-making is robust and interpretable.

8 Results — Competition Leaderboard

8.1 Submission History

8.2 Per-Class OOF F1 (v7 Best Model)

9 Score Decomposition

The final competition score decomposes as:

$$\text{score} = \underbrace{0.5577}_{\text{Macro F1}} \times \underbrace{\max(0.5, 1 - 1.86/200)}_{=0.9907} \times \underbrace{\max(0.5, 1 - 1.18/10)}_{=0.882} = \mathbf{0.4875}$$

Table 21: SHAP Analysis Summary

Plot Type	Key Insight
Bar plot (all 10 classes)	<code>Agency_National</code> dominates; broker encodings collectively are the strongest group
Beeswarm (Class 0)	High <code>Broker_C0</code> values push predictions toward <code>Auto_Comprehensive</code>
Minority class summaries (C8, C9)	These classes rely almost entirely on broker multi-class encoding
Waterfall (individual prediction)	Per-feature contributions with direction and magnitude
Partial dependence plots	Non-linear income thresholds for bundle switching
SHAP interaction analysis	Broker $\times$ Region geographic specialisation patterns
Prediction confidence distribution	Per-class average confidence; <code>Basic_Health</code> is the most confident

Table 22: Complete Submission History

Ver	Key Change	Macro F1	Latency	Score	Lesson Learned
v1	150 trees + threshold tuning	0.4719	2.83 s	0.3331	Thresholds overfit to fold noise
v2	75 trees, no thresholds	0.5176	1.07 s	0.4583	Sweet spot for tree count
v3	50 trees (latency test)	0.4487	0.47 s	0.4253	Too few trees $\rightarrow$ F1 collapse
v4	150 shallow trees depth=4	0.4798	1.23 s	0.4188	Shallow trees underfit
v5	Full-train encoding fix	0.5217	1.03 s	0.4640	Train/inference consistency
v7	All improvements combined	0.5577	1.18 s	0.4875	Best submission

The model achieves near-perfect size penalty while keeping latency well under the 10 s limit. F1 is the dominant lever — the OOF-to-actual gap ($0.6418 \rightarrow 0.5577$) reflects the judge environment’s overhead, not overfitting.

10 What Didn’t Work — Lessons Learned

10.1 Optimising Latency/Size at the Cost of F1

The judge server varies from $3.6\times$ to $10\times$ local latency. **F1 is the only stable lever.**

10.2 `class_weight='balanced'` with SMOTE

Both are imbalance correction mechanisms. Running them together double-penalises the majority class, causing over-prediction of minorities and under-prediction of `Basic_Health`.

Table 23: Per-Class Out-of-Fold F1 Scores

Class	Bundle	OOF F1
C2	Basic_Health	0.846
C9	Renter_Premium	1.000
C5	Home_Premium	0.715
C1	Auto_Liability_Basic	0.674
C7	Premium_Health_Life	0.648
C6	Home_Standard	0.614
C0	Auto_Comprehensive	0.548
C4	Health_Dental_Vision	0.544
C8	Renter_Basic	0.421
C3	Family_Comprehensive	0.408
Macro Average		0.6418

Table 24: Score Decomposition Breakdown

Component	Value	Multiplier
Macro F1	0.5577	—
ZIP size penalty	1.86 MB	$\times 0.9907$
Predict latency penalty	1.18 s	$\times 0.882$
Final score	0.4875	

10.3 Per-Class Threshold Tuning (v1)

Inflated held-out F1 to 0.74 but actual score was 0.3331. Thresholds memorised fold-specific noise rather than generalising. **Dropped in v2.**

10.4 OOF vs. Full-Train Encoding Inconsistency

`train_clean.csv` used OOF target encoding, but `solution.py` used full-train encoding. This mismatch caused the model to see different feature values during training vs. inference. **Fixed in v5 (+0.004 actual F1).**

10.5 Alternative Models Tested

11 Conclusion

11.1 Summary of Contributions

11.2 End-to-End Architecture

Listing 1: Complete Pipeline Architecture

Table 25: Failed Latency Optimisation Attempts

Attempt	Result	Lesson
v3 — 50 trees	F1 dropped 0.07, score fell	Trees < 75 causes F1 collapse for minorities
v4 — 150 shallow trees (depth=4)	F1 = 0.48, score = 0.42	Shallow trees underfit the 10-class problem

Table 26: Alternative Models Evaluated

Model	Issue
CatBoost	Larger model files, no F1 improvement
XGBoost	Comparable F1 but larger serialised files
Random Forest	Too slow and too large for constraints
Deep Learning	Model size exceeds 50 MB ZIP limit

```

1 Raw Data (28 cols)
2   -> Cleaning (drop 3, fill 6, correct 3)
3   -> Feature Engineering (17 new features)
4     -> Encoding (ordinal + binary + one-hot + multi-class target)
5       -> 62-feature matrix
6         -> 3-step resampling (ROS -> SMOTE -> SMOTE)
7         -> LightGBM (75 trees x 10 classes)
8           -> Raw argmax predictions
9           -> Macro F1 = 0.5577, Score = 0.4875

```

11.3 Reproducibility

All code is in self-contained notebooks:

Report generated for ARIA — Insurance Bundle Predictor.

Table 27: Rubric Alignment and Contributions

Rubric Category	Weight	Key Contribution
Feature Engineering	20 pts	17 engineered features + multi-class target encoding (+0.044 F1)
Final Score	30 pts	0.4875 — best of 7 submissions through systematic experimentation
Explainability	10 pts	3 importance methods (SHAP + LightGBM + Permutation) with full visualisation suite
Technical Report	10 pts	This document + notebooks documenting all decisions and trade-offs

Table 28: Repository Structure and Notebook Guide

Notebook	Purpose
<code>data_visualization.ipynb</code>	EDA with 69 cells of analysis
<code>cleaning.ipynb</code>	Data cleaning → <code>train_clean.csv</code> , <code>test_clean.csv</code>
<code>modelling.ipynb</code>	Initial model exploration (v1–v2)
<code>modelling_best_model_v7.ipynb</code>	Best model pipeline + full SHAP explainability
<code>technical_report.ipynb</code>	Consolidated findings and decisions
<code>submission_v7/solution.py</code>	Final production submission code